

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: Генетические алгоритмы

Студенты гр.4345	_____	Д.Н Бабий С.А Кравцов Е.В Гаврилов
Руководитель	_____	Т.Р. Жангиров

Санкт-Петербург
2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студенты: Д.Н Бабий, С.А Кравцов, Е.В Гаврилов

Группа: 4345

Тема практики: Генетические алгоритмы

Задание на практику:

Задача о порядке перемножения матриц

Дано N матриц A с произвольными размерами ($A_1: p_0 \times p_1, A_2: p_1 \times p_2, \dots, A_N: p_{N-1} \times p_N$). Необходимо определить порядок перемножения матриц, чтобы минимизировать количество операций умножения для вычисления результирующей матрицы $B: p_0 \times p_N$

Сроки прохождения практики: 06.07.2026 – 18.06.2026

Дата сдачи отчета: 17.06.2026

Дата защиты отчета: 17.06.2026

Студент

Д.Н Бабий
С.А Кравцов
Е.В Гаврилов

Руководитель

Т.Р. Жангиров

ВВЕДЕНИЕ

Предметной областью данной практики являются методы оптимизации вычислений и эвристические алгоритмы поиска решений. Рассматривается классическая задача минимизации вычислительных затрат, имеющая прикладное значение в различных сферах: от обработки данных до численного моделирования. Особое внимание уделяется генетическим алгоритмам как инструменту решения дискретных оптимизационных задач, где пространство возможных решений велико, а применение точных методов затруднено или нецелесообразно.

Цель практики — закрепление навыков проектирования и реализации программных систем с графическим интерфейсом, а также освоение принципов работы эволюционных алгоритмов через их практическое применение к задаче поиска оптимальной структуры вычислений.

Задачи практики включают изучение теоретического материала, самостоятельную реализацию алгоритмического ядра и визуализации, организацию пользовательского интерфейса для управления ходом вычислений, а также обеспечение возможности анализа и сравнения получаемых результатов. Отдельное внимание уделяется средствам взаимодействия с данными и документированию выполненной работы.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Задача о порядке перемножения матриц актуальна для областей, требующих высокопроизводительных вычислений: машинное обучение, компьютерная графика, САПР, оптимизация запросов в базах данных. Различные варианты расстановки скобок дают одинаковый результат, но количество скалярных операций может отличаться на порядки. Классический метод решения — динамическое программирование. Использование генетического алгоритма в данной работе позволяет исследовать поведение эвристических методов на дискретных оптимизационных задачах, визуализировать процесс эволюции популяции решений и получить навыки реализации алгоритмов без готовых библиотек.

1.2 Задачи практики

1. Изучить математическую постановку задачи перемножения матриц и принципы работы генетических алгоритмов.
2. Выбрать язык программирования, сформировать бригаду, распределить роли.
3. Реализовать вычислительное ядро (функция качества, кодирование решений, операторы селекции, кроссовера и мутации) без использования готовых библиотек ГА.
4. Разработать GUI с возможностью ввода данных вручную, загрузки из файла и случайной генерации.
5. Обеспечить настройку параметров алгоритма пользователем.
6. Реализовать пошаговую визуализацию: текущая аппроксимация, лучшее решение, график функции качества от поколения.
7. Добавить возможность пропуска шагов и перехода к конечному результату.
8. Реализовать откат на несколько шагов назад (доп. баллы).
9. Провести тестирование, сравнение с точным решением, подготовить отчёт.

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Генетический алгоритм

Генетический алгоритм решает задачу минимизации числа скалярных умножений при перемножении цепочки матриц. Решение (особь) кодируется как список целых чисел — на каждом шаге умножения ген указывает индекс матрицы в текущем списке размерностей, которая будет перемножена со следующей. Длина хромосомы равна $N-2$, где N — количество матриц. Такой способ гарантирует корректность любого случайно сгенерированного решения без дополнительной валидации.

Основные компоненты алгоритма:

- `generate_individual` — создаёт случайную особь, где каждый ген g_i лежит в диапазоне $[0, \text{len}-3-i]$.
- `calculate_cost` — вычисляет функцию приспособленности: последовательно перемножает матрицы согласно порядку, заданному хромосомой, и суммирует затраты $p_a \cdot p_b \cdot p_c$ на каждом шаге.
- `calculate_min_cost` — эталонный метод динамического программирования для верификации, возвращает глобальный минимум.
- `tournament` — турнирная селекция: из трёх случайно выбранных особей побеждает особь с наименьшей стоимостью.
- `crossover` — одноточечное скрещивание, порождающее двух детей; корректность потомков обеспечивается одинаковой структурой допустимых значений генов на соответствующих позициях.
- `mutate` — независимая мутация каждого гена с вероятностью p_m , заменяющая ген на случайное допустимое значение.
- `selection` — формирует новое поколение: попарно отбирает родителей, с вероятностью p_c скрещивает, применяет мутацию.
- `genetic_algorithm` — управляющая функция, принимает параметры (размер популяции, число поколений, вероятности операторов,

начальную популяцию и размерности) и возвращает историю поколений в виде списка `GenerationSnapshot` вместе с глобальным минимумом.

- Датакласс `GenerationSnapshot` фиксирует состояние популяции на каждом поколении: номер поколения, лучшую и среднюю стоимость, хромосому лучшего решения и полную популяцию (глубокая копия). Эти данные являются связующим звеном между вычислительным ядром и графическим интерфейсом.

Связь с GUI

Интерфейс получает от пользователя два вида входных параметров:

- 1) размерности матриц заданные вручную(вида `rows x cols`)
- 2) путь к файлу, содержащие размерности
- 3) Количество случайно заданных матриц. После ввода данные передаются в `genetic_algorithm`.

Результатом вызова является список снимков поколений.

GUI использует `GenerationSnapshot` для:

- отображения текущего лучшего решения и его стоимости;
- визуализации порядка перемножения (дерево или последовательность скобок);
- построения графика изменения лучшей и средней стоимости от поколения;
- пошаговой навигации по истории (вперёд, назад, переход к последнему шагу).

Возможность отката на несколько шагов реализуется за счёт сохранения полной популяции в каждом снимке и вызова `genetic_algorithm` с параметром `population`, равным сохранённой популяции требуемого поколения. Это позволяет продолжить эволюцию с любой предыдущей точки без пересчёта всей истории.

Структура компонентов алгоритма и связи алгоритма с графическим интерфейсом приведена на рис. 1 и 2.

2.2. GUI

Графический интерфейс реализован с использованием библиотеки PySide6. Навигация между основными экранами осуществляется с помощью контейнера QStackedWidget, а управление переходами выполняется из MainWindow. Каждая страница и виджет представляют собой независимый класс-наследник QWidget/QGroupBox/QDialog, взаимодействующий с остальной системой посредством механизма сигналов и слотов.

WelcomePage — стартовая страница приложения. Отображает краткое описание назначения программы и предоставляет пользователю возможность перейти к дашборду (next_clicked) или завершить работу приложения (quit_clicked).

DashboardPage — главная страница приложения, в которой осуществляется ввод данных, настройка параметров алгоритма и отображение результатов. На левую панель вынесены источник данных, параметры генетического алгоритма и кнопки управления, на правую панель — график сходимости и результаты текущего шага алгоритма. Содержит кнопки «Запустить алгоритм» и «Значения по умолчанию»; по нажатию первой собирает матрицы и параметры, проверяет их валидность (_validate_input) и запускает расчёт через get_plot_data, после чего обновляет график и результаты.

ChoiceBoard — виджет выбора способа ввода матриц. Пользователь может выбрать один из трёх режимов, переключаемых кнопками:

- случайная генерация заданного количества матриц;
- загрузка данных из текстового файла через QFileDialog;
- ручной ввод размеров матриц через диалоговое окно

ManualInputDialog.

При смене источника интерфейс перестраивается (_on_source_changed): активными становятся только элементы управления, относящиеся к выбранному режиму, а список ранее полученных матриц и статус сбрасываются. Список сформированных матриц отображается в QListWidget,

а статус операции (успех/ошибка) — в текстовой метке с цветовой индикацией. При изменении набора матриц испускается сигнал `matrices_changed`.

ManualInputDialog — диалог ручного ввода цепочки матриц. Позволяет последовательно добавлять размеры (высоту и ширину) очередной матрицы и автоматически проверяет совместимость цепочки на каждом шаге (совпадение соседних размерностей). Предусмотрена возможность удаления последней добавленной матрицы. Кнопка подтверждения («Готово») становится доступна только после добавления не менее двух совместимых матриц; статус-метка подсказывает пользователю текущее состояние («Добавьте матрицы», «Можно считать», «Данные матрицы невозможно перемножить» и т.д.).

GAParamsPanel — панель настройки параметров генетического алгоритма: размер популяции, число поколений, вероятность мутации и вероятность скрещивания. Каждый параметр снабжён подсказкой (`HintLabel`) с всплывающей информацией об его влиянии на работу алгоритма. Предусмотрена кнопка сброса к значениям по умолчанию.

Справа с помощью `pyqtgraph` рисуется график динамики сходимости алгоритма с тремя кривыми: динамика лучшего значения поколения, динамика среднего значения поколения и линия целевого отношения.

ResultsPanel — панель отображения итоговых результатов работы алгоритма: лучшая найденная стоимость, отношение к целевому значению, номер поколения, на котором достигнут лучший результат, и найденный порядок умножения матриц. До получения данных все поля отображают «-».

MainWindow — главное окно приложения.

- создаёт экземпляры `WelcomePage` и `DashboardPage`;
- размещает их в `QStackedWidget`;
- обрабатывает сигналы навигации (`next_clicked`, `quit_clicked`), определяя переход между приветственной страницей и дашбордом;
- устанавливает стартовую страницу и общие параметры окна (заголовок, размер).

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

4 ОТЗЫВ РУКОВОДИТЕЛЯ

Если прохождение практики было по кафедральным темам достаточно добавить формулировку:

По результатам работы учебная практика оценена на <ваша_оценка>.

Если прохождение практики было по своей теме, то необходимо добавить скан отзыва руководителя с оценкой и подписью, скан должен быть в высоком качестве на всю страницу.

Для прохождения автоматической проверки для скана необходима подпись и ссылка в тексте, т.к. он считается рисунком.

Отзыв руководителя с оценкой (см. рис. X).

Отзыв

о прохождении учебной практики

<ФИО студента>, студент(ка) группы <номер группы>, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил(а) учебную практику по теме “<тема практики>” с <дата начала> по <дата окончания>.

В течение практики обучающийся(аяся) <ФИО студента> выполнял <задание на практику>.

В результате практики обучающийся(аяся) <результат практики>. Получаемую работу обучающийся(аяся) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебной практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата>

<подпись>/<ФИО>

Рисунок X — Отзыв руководителя

ЗАКЛЮЧЕНИЕ

Заключение содержит краткое описание результатов по каждой задаче, обозначенной в разделе «Постановка задачи», результаты работы в целом, согласно обозначенной цели и пр.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. The State of the Octoverse [Электронный ресурс]. URL: <https://octoverse.github.com> (дата обращения: 28.04.2021).
2. Бобкова, О.В., Давыдов, С.А., Ковалева, И.А., Плагиат как гражданское правонарушение / О.В. Бобкова, С.А. Давыдов, И.А. Ковалева // Патенты и лицензии. – 2016. – № 7. – С. 31-37.

ПРИЛОЖЕНИЕ А
ДИАГРАММА КЛАССОВ

Matrix Chain GA — GUI to Backend Collaboration

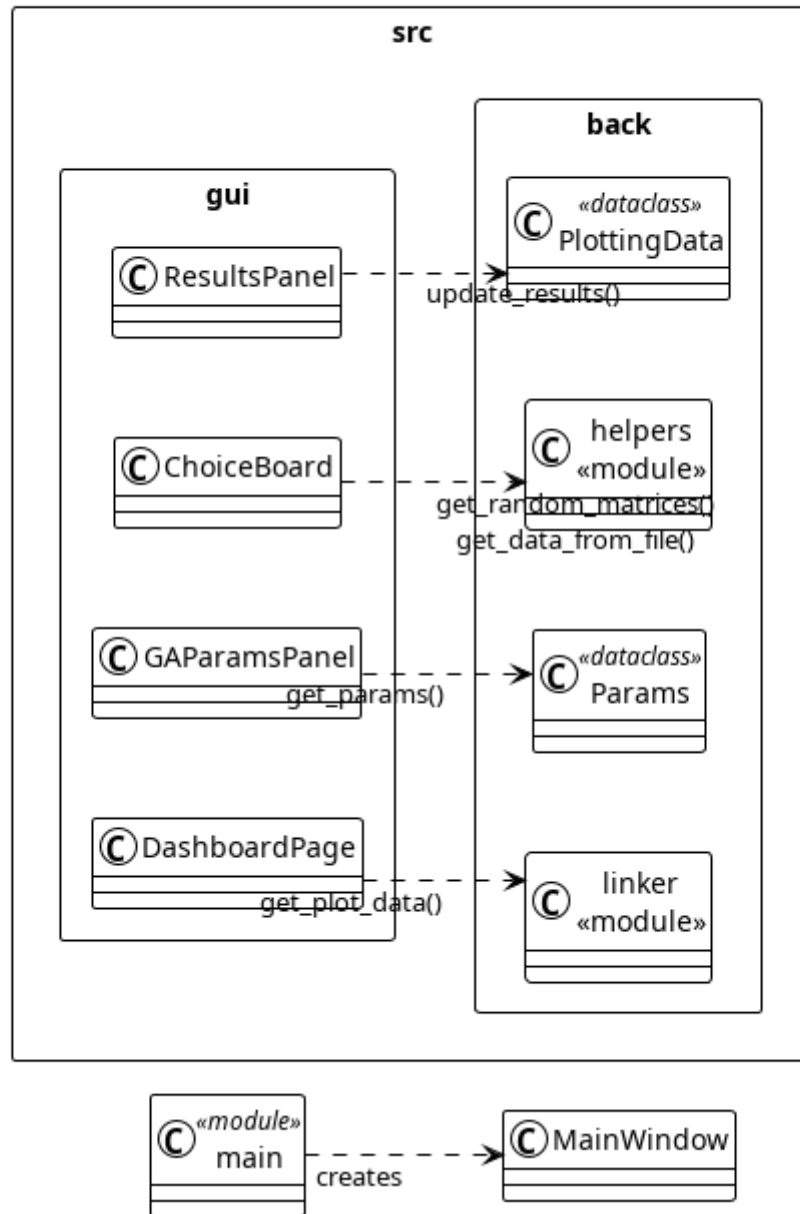


Рисунок 1 — Диаграмма классов связи GUI и backend

Matrix Chain GA — Backend Structure (src.back)

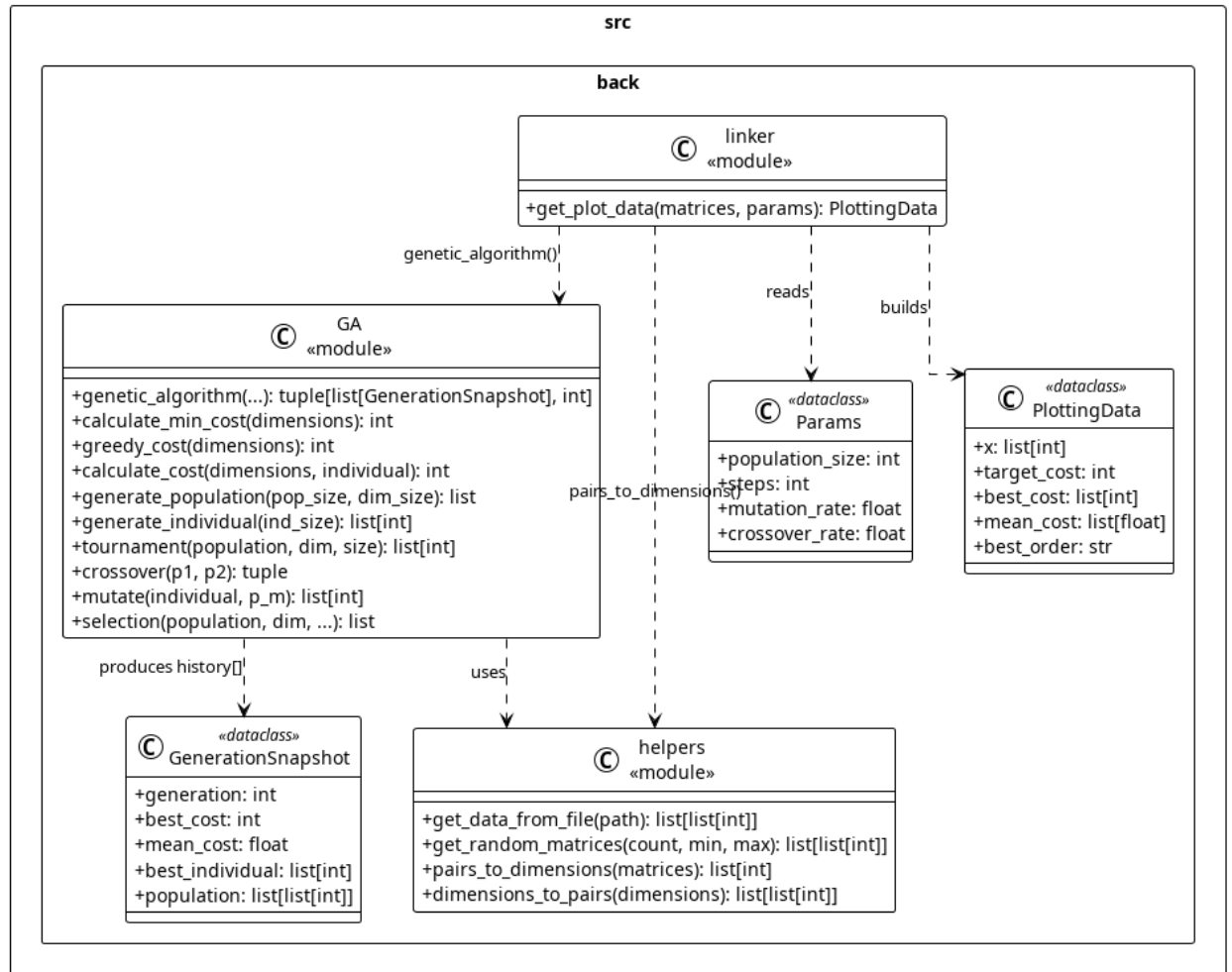


Рисунок 2 — Диаграмма классов backend